Exemplo de Exercícios para a Prova - Grafos

Questão 1. A função de impressão de grafos vista em aula foi chamada para imprimir um grafo direcionado e ponderado representado por listas de adjacências. O resultado do que foi impresso pode ser visto a seguir. Desenhe o **grafo direcionado e ponderado** correspondente.

Imprimindo grafo (vertices: 6; arestas: 10).
[0] -> 0 (1.00) -> 3 (3.50)
[1] -> 0 (6.00) -> 4 (5.00)
[2] -> 1 (2.50) -> 3 (2.00) -> 4 (6.00)
[3] -> 5 (4.00)
[4] -> 5 (3.00)
[5] -> 2 (4.50)

Desenhe o grafo aqui:

Questão 2. Dado o seguinte grafo **ponderado e não direcionado**, representado pela matriz de adjacências com pesos apresentada a seguir, desenhe a **árvore geradora de custo mínimo** para esse grafo.

	v0	v1	v2	v3
v0	-	5.00	12.00	-
v1	5.00	-	9.00	10.00
v2	12.00	9.00	-	4.00
v3	-	10.00	4.00	-

Desenhe a árvore geradora de custo mínimo aqui:

Questão 3. Considerando a representação de grafos **não ponderados e não direcionados** usando **matrizes de adjacências booleanas**, com base na estrutura apresentada a seguir (que é a mesma que foi vista em aula), implemente as duas funções solicitadas.

```
typedef struct {
  int numVertices;
  int numArestas;
  bool** matriz;
} Grafo;
```

a) verticeMaiorGrau: função que recebe o endereço de um grafo (g) e retorna o número/identificador do vértice que tem o maior grau no grafo (se mais de um vértice tiver o maior grau, sua função deve retornar o menor deles [aquele com o menor identificador]). Você pode considerar que g contém um endereço válido para um grafo. **Não use nenhuma função auxiliar.**

```
int verticeMaiorGrau(Grafo* g){




}
```

b) vizinhanca2: função que recebe o endereço de um grafo (g) e o identificador de um vértices (u) e retorna o número total de vértices que são vizinhos de u ou vizinhos dos vizinhos de u (isto é o número de vizinhos de u, mais o número de vizinhos dos vizinhos de u [cuidado para não contar um mesmo vértice duas vezes e você não deve incluir u na contagem). Você pode considerar que g contém um endereço válido para um grafo.

```
int vizinhanca2(Grafo* g, int u){




}
```

Questão 4. O algoritmo de Dijkstra, que calcula o arranjo de distância e de predecessores de um vértice para os demais vértices de um grafo, foi executado para um dado grafo, a partir do vértice 0 (zero) e produziu os seguintes arranjos de distância e de predecessores:

Exibindo arranjo de distâncias.

v0	v1	v2	v3	v4	v5
0.00	3.50	1.00	3.00	7.00	7.00

Exibindo arranjo de predecessores.

v0	v1	v2	v3	v4	v5
0	2	0	2	2	3

Com base nesses arranjos, escreva qual será o caminho de menor custo (escreva a sequência de vértices na ordem em que serão percorridos) e o custo (ou distância) total para:

a) Sair do vértice **v0** e chegar no vértice **v4**:

Caminho: Distância:

b) Sair do vértice **v0** e chegar no vértice **v5**:

Caminho: Distância:

c) Sair do vértice **v0** e chegar no vértice **v3**:

Caminho: Distância:

Questão 5. O algoritmo de Floyd-Warshall, que calcula as distâncias (e os predecessores) entre todos os pares de vértices, foi executado para um dado grafo (ilustrado a seguir pela impressão do grafo no formato de listas de adjacências) e produziu as seguintes matrizes de distâncias e de predecessores:

Imprimindo grafo (vértices: 6; arestas: 13).
[0] -> 3 (10.00) -> 5 (76.00)
[1] -> 2 (21.00) -> 4 (14.00)
[2] -> 2 (73.00) -> 5 (44.00)
[3] -> 1 (57.00) -> 2 (70.00) -> 5 (78.00)
[4] -> 1 (16.00) -> 3 (55.00)
[5] -> 0 (55.00) -> 4 (47.00)

Exibindo matriz de distâncias.

	v0	v1	v2	v3	v4	v5
v0	0.00	67.00	80.00	10.00	81.00	76.00
v1	120.00	0.00	21.00	69.00	14.00	65.00
v2	99.00	107.00	0.00	109.00	91.00	44.00
v3	133.00	57.00	70.00	0.00	71.00	78.00
v4	136.00	16.00	37.00	55.00	0.00	81.00
v5	55.00	63.00	84.00	65.00	47.00	0.00

Exibindo matriz de predecessores.

	v0	v1	v2	v3	v4	v5
v0	0	3	3	0	1	0
v1	5	1	1	4	1	2
v2	5	4	2	0	5	2
v3	5	3	3	3	1	3
v4	5	4	1	4	4	2
v5	5	4	1	0	5	5

a) Qual é a menor distância entre o vértice v0 e o vértice v4? _____

b) Qual vértice está mais distante de v2? _____

c) Qual é o par de vértices mais distante (isto é, se você precisar sair de um vértice x e chegar a um vértice y, qual é o par x e y cujo caminho tem a maior distância)?

d) O grafo desta questão é cíclico ou acíclico? _____

e) O grafo desta questão é fortemente conexo? _____

f) Desenhe a Árvore de Caminhos de Menor Custo (ou de Custo Mínimo) do vértice v3 (isto é, aquela em que v3 é a raiz):	**f)** Desenhe a Árvore de Caminhos de Menor Custo (ou de Custo Mínimo) do vértice v4 (isto é, aquela em que v4 é a raiz):

Questão 6. Considerando os quatro "tipos" de arestas potencialmente identificadas durante a busca em profundidade em grafos direcionados (aresta de árvore, de retorno, de avanço e de cruzamento).
Que características definem cada uma destas arestas?
Aresta de árvore: (exemplo de resposta) é a aresta (x,y) pela qual, na busca em profundidade, se visita o vértice y pela primeira vez. As arestas de árvore são as arestas que estarão presentes nas Árvores de Busca em Profundidade.

Aresta de Retorno:

Aresta de Avanço:

Aresta de Cruzamento:

Questão 7. Em um grafo direcionado que possui arestas de retorno é possível realizar uma ordenação topológica?

Questão 8. Considerando os diferentes quatro tipos de arestas da questão anterior, qual (ou quais) destas arestas sempre estará(ão) presente(s) em um grafo direcionado **cíclico**?

Questão 9. Para cada alternativa assinale **V** para verdadeiro ou **F** para falso. **Cada alternativa assinalada incorretamente anulará uma alternativa assinalada corretamente**, na dúvida, sugere-se deixar a alternativa em branco.

( ) Um grafo não direcionado conexo sempre possui pelo menos uma árvore geradora.

( ) Um grafo não direcionado com n vértices e n arestas é necessariamente um grafo completo.

( ) O algoritmo de Kruskal sempre constrói uma árvore geradora mínima se o grafo for não direcionado e conexo e ponderado.

( ) O algoritmo de Dijkstra funciona corretamente mesmo quando o grafo possui arestas de peso negativo.

( ) Em um grafo dirigido, uma busca em profundidade pode identificar ciclos.

( ) A busca em largura pode ser usada para encontrar o caminho mais curto (menor número de arestas) entre dois vértices em grafos não ponderados.

( ) Uma árvore geradora mínima de um grafo é única, independentemente dos pesos das arestas.

( ) O algoritmo de Prim e o de Kruskal podem produzir árvores geradoras diferentes, mas com o mesmo custo total.

( ) Um grafo com pesos negativos pode ser tratado com o algoritmo de Bellman-Ford para encontrar caminhos mínimos, desde que não possua ciclos negativos.

( ) A complexidade da busca em largura em um grafo representado por lista de adjacência é $O(V + E)$.

( ) Em um grafo não dirigido e conexo, toda árvore geradora é uma subárvore que inclui todos os vértices e é acíclica.

( ) Um grafo com n vértices pode ter no máximo $n-1$ arestas.

( ) O algoritmo de Dijkstra pode usar uma fila de prioridade para escolher o próximo vértice a ser processado.

( ) Em um grafo não direcionado com arestas ponderadas, a busca em largura encontra o caminho de menor custo entre dois vértices.

( ) O algoritmo de Bellman-Ford pode detectar ciclos negativos em grafos dirigidos.